DashNode: Technical Specification & System Architecture

Undergraduate IT Engineering Capstone Project | Modular Vehicle Telemetry Platform

---

1. Executive Summary & Problem Definition

Modern vehicle instrument clusters frequently lack configurable, high-precision telemetry such as continuous intake air temperature, true oil temperature, or dynamic ECU bus parameters. Commercially available auxiliary gauges often introduce significant cabin clutter through intrusive physical wiring harnesses, while generic smartphone applications suffer from slow manual Bluetooth connectivity and severe battery drain. **DashNode** introduces an open-source, split-architecture automotive telemetry platform that cleanly decouples high-speed data acquisition from dashboard visualization.

The acquisition node directly interfaces with the legislated J1962 (OBD-II) port under the footwell, running an automotive-grade buck regulator and an integrated CAN transceiver. Telemetry frames are filtered, parsed, and broadcasted over a dedicated low-latency 2.4 GHz wireless link (ESP-NOW) to an auxiliary compact circular gauge node mounted in the driver's visual line-of-sight.

2. Hardware Subsystems & Component Specifications

The physical architecture is partitioned into two independent hardware modules to isolate automotive power transients from the user interface:

Subsystem	Component / Controller	Key Engineering Characteristics
Acquisition MCU	ESP32-C3-WROOM-02	Single-core 32-bit RISC-V @ 160MHz, built-in TWAI (CAN 2.0B), integrated 2.4 GHz RF.
Step-Down Buck	TPS54302 / MP2315	Input 4.5V–28V, synchronous switching @ 400kHz–500kHz, TVS & Schottky protection.
CAN Transceiver	SN65HVD230 / TJA1051T/3	3.3V logic compatibility, ISO 11898 compliant, optional switchable 120Ω bus load.
Display MCU	ESP32-S3-WROOM-1	Dual-core Xtensa LX7, 8MB Octal PSRAM, dedicated Core 1 hardware graphics processing.
Display Panel	1.28" Round IPS TFT (GC9A01)	240x240 circular active area, 4-wire high-speed SPI with DMA support, CST816S capacitive touch.

3. Firmware Architecture & Protocol Stack

A. Acquisition Loop: The ESP32-C3 TWAI engine initializes at 500 kbps (standard 11-bit ISO 15765-4). A hardware timer schedules cyclic OBD-II PID query frames (Mode 01): RPM (0x0C) and Speed (0x0D) polled at 10–20 Hz, while Coolant Temp (0x05) and Ambient/Intake Temps (0x0F) are queried at 1 Hz to preserve bus bandwidth.

B. Wireless Peer-to-Peer Pipeline: Data frames are packed into a compact 16-byte binary struct and broadcasted using ESP-NOW. This connectionless protocol bypasses Wi-Fi handshake overhead, delivering telemetry packet latencies under 5 ms.

C. Dual-Core Display Execution: The ESP32-S3 allocates tasks across separate processing cores to prevent GUI stutter:

• **Core 0:** Wireless telemetry packet reception, checksum verification, and optional BLE media background services.
• **Core 1:** Light and Versatile Graphics Library (LVGL) render engine refreshing circular needle animations and digital readouts.

4. Implementation Milestones & Risk Control

Phase / Milestone	Duration	Key Deliverables & Verification Criteria
Phase 1: Proof-of-Concept	Weeks 1–2	Breadboard CAN transceiver verification; ESP-NOW packet link; basic LVGL gauge render.
Phase 2: Custom Hardware	Weeks 3–4	KiCad schematic capture; buck regulator calculation; 2-layer PCB layout; fab order release.
Phase 3: Integration	Weeks 5–6	SMD assembly; power rail validation (12V to 3.3V); vehicle live bus testing; enclosure print.
Phase 4: Defense & Thesis	Weeks 7–8	Thesis documentation; dynamic latency bench measurements; final university presentation.